Current Prediction in Analog Integrated Circuits Using Graph Convolution Networks

Matthew Jones
Oregon State University
Corvallis, Oregon
jonesm25@oregonstate.edu

Alexander La Chapelle
Oregon State University
Corvallis, Oregon
lachapea@oregonstate.edu

Jonathan Guzman
Oregon State University
Corvallis, Oregon
guzmanjo@oregonstate.edu

Abstract

Modern place and route agents for analog integrated circuits are required to meet the design rules. For many interconnects this may be overkill, leading to wasted area. Determining which interconnects can be drawn with strategic design rule violations, without sacrificing reliability, requires knowledge of the current flowing through the interconnect. Running simulations on the full analog design is expensive. This paper proposes a simpler way: A Deep Learning model that can predict the current based on the design netlist and manufacturing process models. Results do not show that our network is sufficiently accurate, the convergence of our training algorithm suggests that, predicting current from a netlist and graph structure could still be possible with further performance improvements.

1. Introduction

The motivation behind this project is to further research into the use of Deep Learning to minimize analog integrated circuit design area without sacrificing reliability. A recent review of the literature searching All Metadata for "Artificial Intelligence" or "Reinforcement Learning" or "Machine Learning," and "Design Rule" or "Layout" or "Route," in the ISSCC, the JSSC, the CICC, or the Symposium on VLSI Circuits resulted in only 10 papers in the last 10 years. Park twice wrote about ML-based layout optimization to control timing skews [11] [12] and Zhou wrote about using RL for design space discovery and end-to-end synthesis [16]. However, most wrote about IC designs and methodologies to improve AI model performance, such as Dong [3], Kaul [7], Everson [5], Zhao [14], Feng [6], Sinangil [13], and Cho [2]. None of the ten papers found in these top journals were about Deep Learning, IC Area, or Reliability. The long-term goal of this project is to change that. The short-term goal of the project is to prove it can be done.

1.1. Physical Design

Even people who are unfamiliar with the integrated circuit design process are familiar with the concept of a circuit schematic, which is a symbolic representation of the circuit's function. Physical design is the process of "drawing" the circuit elements how they are actually put on silicon. Individual circuits may be designed and used as "blocks" in other circuits. Connecting blocks within a larger circuit is the process of "placing" the blocks and then "routing" the connection wires between them. See Figure 1 for an example of a basic two-block circuit schematic and the corresponding physical design.

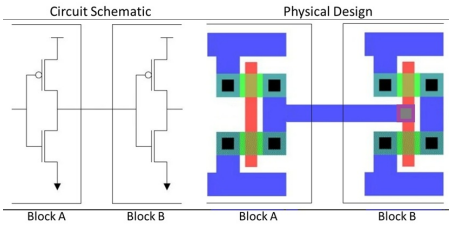

Figure 1: Circuit Example

Typically, these metal wires are drawn to meet the minimum width requirements in the design rules for the manufacturing process. However, if a design rule (DR) must be violated to avoid rework or save area, a waiver for the rule may be granted. Design rules can be overkill for metal lines with little current flowing through them. However, typical Reinforcement Learning-based place and route (P&R) agents are constrained by the design rules. Strategic design rule violations could potentially provide significant area savings without a significant impact on reliability. The key is to know which connection wires can violate the rules without sacrificing reliability.

1.2. Strategic DR Violation

The solution is to provide the Reinforcement Learning (RL) agent with the current flowing through each interconnection wire so that it can decide, based on current design calculations, which wires can be made narrower than the design rule. Unfortunately, creating a simulation testbench for each circuit is expensive. A solution to this expense is to use a Deep Learning model to estimate the current based on the design and process parameters, provide that information to the RL agent which then performs P&R, then calculate the circuit reliability based on the routed layout. This flowchart is shown in Figure 2.

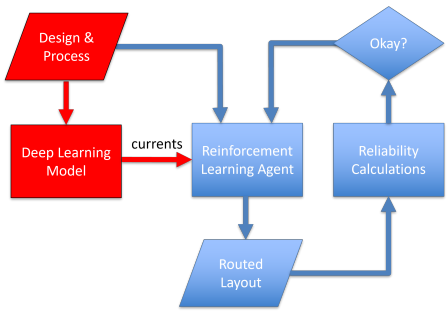

Figure 2: System Flowchart

The Deep Learning model needs to be trained on a dataset that contains simulation results representative of a circuit testbench. The remainder of this report describes how this Deep Learning model was design and trained, and is organized as follows: The next section describes dataset generation process, including the circuit description, the simulation methodology, and the datasets that were created. Section 3 describes the Deep Learning Network, including the details of the Graph Neural Network and the Training results. The Conclusion is Section 4.

1.3. Graph Neural Networks

Graph neural networks are a type of neural network that uses a complete graph structure, rather than a single data sample, as one input. Mathematically, the graph is represented as a weighted adjacency matrix and a feature matrix whose rows are the feature vectors of each individual node in the graph. Learning can then be done to do classification or regression on node labels, edge weights, or an aggregate graph level [15]. GNNs can consist of many different types of layers, but the one that is most central to the concept is known generally as the message passing layer. This layer updates the features at each node, using information pulled from neighboring node features. For a general GNN this is not strictly defined, although the edges typically encode how the 'messages' are passed between nodes.

While there are many types of GNN, Graph Convolution Networks (GCNs) are a good starting point for our experiment due to their simple, yet rigorously defined message passing step, which transforms the feature matrix via multiplication with the adjacency matrix [8]. GCNs are easily applicable in problems where some sort of topological adjacency exists between elements, especially if that adjacency describes a model-able interaction. In our case the application is fairly obvious, with localized circuit properties encoding how current flow is moving through wires in a particular simulation snapshot.

1.4. Related Work

There is already a large amount of existing literature on the topic of GNNs. For circuits specifically, El Sayed et al [4] conduct a survey on GNNs as tools in circuit design, although their scope is considerably broader than our own, discussing not just physical design but other applications such as logic design and synthesis.

For direct application, Mirhoseini et al use edge-based GCNs to solve optimization problems in physical design [10]. Their work once again differs from ours in the sense that the scope is much broader. Additionally, their approach differs in that they use deep reinforcement learning, while we used a simpler supervised learning approach.

2. Dataset

The first step toward creating a dataset is to choose a test circuit. For this project, the test circuit is a Cascaded CMOS Inverter Chain, as shown below. The target node, which represents the connection between two circuit blocks, is between them.

Functionally, it is as simple as the circuit shown in Figure 1, but requires additional elements related to the testbench in order to capture the simulation results necessary to generate the dataset. The description of each element is below.

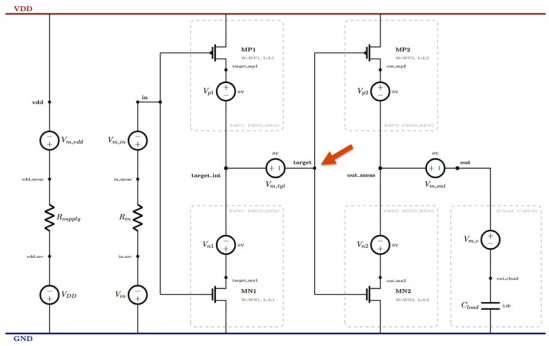

Figure 3: Cascaded Inverter Testbench (2inv.sp)

2.1. Circuit Description

The testbench is comprised of transistors, subcircuits, measurement sources, and supply and input sources. The transistors are what make up the inverters. The rest of the elements are to facilitate measurement for data capture. A description of each element is below.

Transistors

- *MP1* (PMOS, W=WP1, L=L1) is the pull-up transistor of the *driver inverter* (INV1). When the input is low, MP1 charges the `target` node toward VDD. Its width WP1 and length L1 are swept across the design space.
- *MN1* (NMOS, W=WN1, L=L1) is the pull-down transistor of the driver inverter. When the input is high, MN1 discharges `target` toward ground. Together with MP1, it sets the switching threshold and transition speed of INV1.
- *MP2* (PMOS, W=WP2, L=L2) is the pull-up transistor of the *load inverter* (INV2). Driven by the `target` node, it charges the `out` node. Its dimensions WP2 and L2 are independently swept.
- *MN2* (NMOS, W=WN2, L=L2) is the pull-down transistor of the load inverter. Together with MP2, it presents a realistic capacitive and resistive load to the driver inverter, which directly affects the peak current drawn from VDD.

Subcircuits

- *XMP1: PMOS_MEAS* (contains MP1 + V_{p1}) wraps the PMOS transistor with a 0 V measurement voltage source (V_{p1}) in series with its drain. This allows SPICE to report the instantaneous drain current of MP1 as a branch current without disturbing the circuit.
- *XMN1: NMOS_MEAS* (contains MN1 + V_{n1}) uses the same technique for the NMOS transistor of INV1, enabling measurement of MN1's drain current via V_{n1}.
- *XMP2: PMOS_MEAS* (contains MP2 + V_{p2}) is the rain current measurement wrapper for MP2 in the load inverter.
- *XMN2: NMOS_MEAS* (contains MN2 + V_{n2}) is the rain current measurement wrapper for MN2 in the load inverter.
- *XCload: C_MEAS* (contains C_{load} + $V_{m,c}$) wraps the 5 fF load capacitor with a 0 V measurement source. This captures the current flowing into the capacitive load, representing the dynamic charging/discharging current that a downstream stage would see.

Measurement Sources (0 V)

- $V_{m,vdd}$ (Vmeas_vdd) measures the total current drawn from the VDD supply rail. Its peak value (`I_vdd`) captures supply current behavior relevant to power integrity and IR-drop risk.
- $V_{m,in}$ (Vmeas_in) measures the current delivered by the input source. Captures gate-tunneling and capacitive charging currents at the input node (`I_in`).
- $V_{m,tgt}$ (Vmeas_target) measures the current flowing from the driver inverter's output node (`target_int`) to the load inverter's gate (`target`). Records `I_target`, the primary prediction target for the DL model, reflecting the inter-stage drive current.
- $V_{m,out}$ (Vmeas_out) measures the current at the output of the load inverter (`I_out`), capturing the current flowing into the load capacitor path.
- V_{p1}, V_{n1}, V_{p2}, V_{n2} is internal to PMOS_MEAS / NMOS_MEAS subcircuits. Each measures the drain current of its respective transistor, enabling per-device current attribution.
- $V_{m,c}$ is internal to C_MEAS. Measures the capacitor charging current, distinguishing it from resistive leakage paths.

Supply and Input Sources

- V_{DD} is the DC voltage source providing the supply voltage. The value is swept across the design space to capture voltage-dependent behavior.

- R_{supply} is a small series resistance (0.01 Ω) in the VDD path, modeling interconnect resistance and aiding convergence.
- V_{in} is the pulsed voltage source (PULSE) generating the switching stimulus. Rise/fall times are 20 ps; pulse width is 1 ns with a 2 ns period.
- R_{in} is a small series resistance (0.01 Ω) in the input path, matching the VDD path topology.

The goal of the simulation is to measure the max voltage and peak current at each node in circuit, so it can be added to the dataset. The nodes are described below.

- *vdd* is a regulated supply rail, downstream of $V_{m,vdd}$. Voltage `V_vdd` is recorded to verify supply integrity; current through $V_{m,vdd}$ gives the supply current target.
- *in* is the gate input to INV1, downstream of $V_{m,in}$. Driven by a PULSE source (V_{in}) through R_{in}. Voltage `V_in` and current `I_in` are recorded.
- *target_int* is an internal output of INV1 (junction of MP1 and MN1 drain paths), before the measurement source $V_{m,tgt}$.
- *target* is the gate input to INV2, downstream of $V_{m,tgt}$. Voltage `V_target` and current `I_target` are recorded. This is the inter-stage node whose transition drives the peak supply current.
- *out_meas* is an internal output of INV2 (junction of MP2 and MN2 drain paths), before $V_{m,out}$.
- *out* is the final output node, downstream of $V_{m,out}$. Voltage `V_out` and current `I_out` are recorded.
- *GND* (node 0) is the global ground reference. `I_gnd` is derived by Kirchhoff's current law as the negative sum of all other node currents.

Table 1 below shows the simulation results that will be captured as features in the dataset.

2.2. Simulation Methodology

The dataset was generated by simulating, using ngspice, the circuit testbench across different process models, process corners and skews, and transistor sizes. Each simulation represents a unique combination of process technology, operating conditions, device asymmetry, and circuit parameters. These variations are described below.

Process Models (9 variations) [9]. The simulation covers nine different semiconductor process technologies:

- *22nm HP:* Low threshold voltage, thin oxide for high performance 22nm node
- *22nm LP:* Higher threshold voltage for reduced leakage and low power 22nm node
- *32nm HP:* High performance 32nm node
- *32nm LP:* Low power 32nm node
- *45nm HP:* High performance 45nm node
- *45nm LP:* Low power 45nm node
- *65nm Bulk:* Traditional CMOS bulk process (neither HP nor LP)
- *90nm Bulk:* Mature bulk technology (neither HP nor LP)
- *130nm Bulk:* Legacy bulk process (neither HP nor LP)

PVT Corners (5 variations). Process, Voltage, Temperature combinations for comprehensive operating conditions:

- *Typical:* Nominal operating conditions
- *Slow:* Conservative: Low voltage, high temperature
- *Fast:* Aggressive: High voltage, low temperature
- *Hot:* High temperature stress testing
- *Cold:* Low temperature operation

Process Skew Corners (5 variations). Device asymmetry variations capturing realistic NMOS/PMOS device mismatches:

- *TT:* Balanced device performance
- *FF:* Both devices fast (high performance)
- *SS:* Both devices slow (low leakage)
- *FS:* Asymmetric: affects rise vs fall times
- *SF:* Asymmetric: affects fall vs rise times

Monte Carlo Device Parameters. For each combination above, the following parameters are randomly sampled:

- *WN1, WN2:* NMOS transistor widths (process-specific ranges)
- *WP1, WP2:* PMOS transistor widths (process-specific ranges)
- *L1, L2:* Channel lengths, 20nm-600nm (node dependent)
- *VDD:* Supply voltage, 0.7V-2.0V (node dependent)
- *TEMP:* Operating temperature, -20°C to 125°C
- *Example 22nm ranges:* W: 0.2μm - 8μm, L: 20nm - 200nm

Feature	Source	Description
V_vdd	$V_{m,vdd}$, node vdd	Peak voltage at VDD rail
V_in	$V_{m,in}$, node in	Peak voltage at input
V_out	$V_{m,out}$, node out	Peak voltage at output
V_gnd	node 0	Ground reference (always 0)
V_target	$V_{m,tgt}$, node target	Peak voltage at inter-stage node
I_vdd	$V_{m,vdd}$ branch	Peak supply current
I_in	$V_{m,in}$ branch	Peak input current
I_out	$V_{m,out}$ branch	Peak output current
I_gnd	Kirchhoff (derived)	Ground return current
I_target	$V_{m,tgt}$ branch	**Peak inter-stage current (prediction target)**

Table 1: Simulation Results as Dataset Features

2.3. Dataset 1

Dataset 1 was generated using only a single process model, 22nm_HP.pm. This was to ensure that the model could be trained on a smaller feature size. This means that for every combination of device parameters, 25 (5 corners x 5 skews) simulations were run. Figure 4 shows a single instance of dataset 1. The dataset contains 100,000 instances.


```
{
  "ID": 0,
  "Design": "2inv",
  "Size": "22",
  "Option": "HP",
  "PVT": "typical",
  "Skew": "SS",
  "WN1": 3.987860998114745e-06,
  "WP1": 7.346739531762215e-06,
  "WN2": 3.0587266286604007e-06,
  "WP2": 3.038069119888764e-06,
  "L1": 2.762855924312858e-08,
  "L2": 1.228588215527502e-07,
  "VDD": 0.8906585278880905,
  "Temp": 40.103664373134855,
  "V_vdd": 0.8906628,
  "V_gnd": 0.0,
  "V_in": 0.8906585,
  "V_out": 1.205001,
  "V_target": 0.9205885,
  "I_vdd": 0.00193829,
  "I_gnd": -0.0032357769343999998,
  "I_in": 0.0009869179,
  "I_out": 0.0003105529,
  "I_target": 1.61344e-08
},
```


Figure 4: Dataset 1 in JSON Format

ID is the instance number. Design is the circuit. We wanted to have that in the dataset, even though it's the same for every instance because at some point we want to test this with multiple circuits. Size and Option are the process. 22 is the minimum feature size in nanometers and HP means it's the High Performance version of the model. This is as opposed to LP, which stands for Low Power. PVT and Skew are simulation corners. PVT is Process, Voltage, and Temperature. The voltage is the supply voltage. Skew is how the transistors perform. SS means both N-type and P-type are slow. Skew can also be fast/fast, fast/slow, slow/fast, and typical/typical. The Ws and Ls are transistor sizes. For example, WN1 is the gate width of the N-type transistor in the first stage. Similarly for the others. It's typical for the N-type and P-type transistors to have the same gate length, so there are only two lengths. VDD is the supply voltage and Temp is the temperature. The rest are the voltages and current for the nodes, the last being the target current the model is going to predict.

Dataset 1 can be found here: `https://web.engr.oregonstate.edu/~jonesm25/datasets/dataset.json`.

2.4. Dataset 2

Dataset 2 was generated using all 12 process models. This was to ensure that the model could be flexible. This means that for every combination of device parameters, 25 (5 corners x 5 skews) simulations were run. Figure 5 shows a single instance of dataset 2. The dataset contains 112,500 instances.

The second dataset has all of the same features as the first dataset, plus some more. The additional features are parameters that change between process models. The parser automatically removes any parameter that doesn't change at all. Some parameters don't change much, so they may need to be pruned. Also, because some of the numbers are huge and time are minuscule, they may need to be normalized. Table 2 defines the new features in Dataset 2. Note that they have _N and _P versions. This is because the parameters differ for N-type and P-type transistors.

Dataset 2 can be found here: `https://web.engr.oregonstate.edu/~jonesm25/datasets/dataset2.json`.


```
"VTH0_N": 0.3782,
"TOX_N": 2.25e-09,
"TOXP_N": 1.6e-09,
"TOXM_N": 2.25e-09,
"U0_N": 0.05928,
"UC_N": 0.0,
"VSAT_N": 100370.0,
"XJ_N": 3.92e-08,
"NDEP_N": 1.54e+18,
"NF_N": 1.5,
"ETA0_N": 0.0092,
"K2_N": 0.01,
"DVT2_N": -0.032,
"VOFF_N": -0.13,
"PCLM_N": 0.06,
"RDSW_N": 200.0,
"CGSO_N": 2.4e-10,
"CGDO_N": 2.4e-10,
"VTH0_P": -0.321,
"TOX_P": 2.35e-09,
"TOXP_P": 1.6e-09,
"TOXM_P": 2.35e-09,
"U0_P": 0.00835,
"UC_P": -3e-11,
"VSAT_P": 70000.0,
"XJ_P": 3.92e-08,
"NDEP_P": 1.14e+18,
"NF_P": 1.5,
"ETA0_P": 0.0092,
"VOFF_P": -0.126,
"RDSW_P": 240.0,
"CGSO_P": 2.4e-10,
"CGDO_P": 2.4e-10,
```


Figure 5: Part of Dataset 2

Parameter(s)	Meaning
VTH0	Zero-bias threshold voltage ("on" voltage)
VOFF	Adjusts the subthreshold current curve so simulation matches measured data
TOX / TOXP / TOXM	Gate dielectric thickness, which determines gate capacitance
U0	Controls carrier mobility, which determines current drive
UC	Mobility degradation coefficient
VSAT	Carrier velocity saturation limit
XJ	Junction depth of source/drain diffusion
NDEP	Channel doping concentration
DVT2	Second-order short-channel threshold roll-off parameter
ETA0	Controls how drain voltage lowers the threshold voltage
K2	Second-order body effect coefficient
PCLM	Controls channel length modulation
RDSW	Source/drain sheet resistance per width
CGSO / CGDO	Gate-to-source overlap capacitance
NF	Number of gate fingers

Table 2: Simulation Results as Dataset Features

3. Technical Approach

The technical approach we took was to implement a Graphical Convolution Network (GCN).

3.1. GCN Architecture

To predict current values, we use regression on the graph's edge weights with a GCN. For this problem, we have designed the graphs such that our training is entirely supervised.

Because the graph models for this experiment have a simple structure, many of the relevant literature discusses improvements to the GCN architecture that are beyond our need at this time. As a result, we have mainly determined the performance changes from architectural changes experimentally. What we have found, generally, is that the introduction of wider or deeper networks has had limited results. Our final network consists of three graph convolution layers, although future work could be done to determine if additional layers are useful under other circumstances.

3.2. Graphical Circuit Models

Creating a graphical model of a circuit can be approached in several different ways. In physics problems, a circuit is typically translated to graph form by treating partitions of wire with equal potential as the nodes. For our case, however, we prefer to encode circuit components as nodes to limit feature data to nodes and label data to edges, respectively. Additionally, we can model the individual inverters as either a black-box node, or split it into two nodes with individual connections.

Ideally, we would like two nodes to be directly connected if they are connected via a wire in the real circuit. However, wires are not exactly analogous to an edge in a standard graph, due to being able to connect several nodes at once. To be physically accurate, we would need to model all of the parallel circuits this creates, but our dataset currently leaves these wires with unknown currents that can't constrain our loss function. At this point we leave physically accurate modeling to future work, and implement our models with duplicated currents, using the black-box and split inverter styles.

3.3. Loss Function

One problem we encountered during training is that some currents in the dataset (the main target current specifically) is relatively small. For the purpose of eventual generalization, we still aim to predict all edge weights in a graph, so graph-level prediction is

not an option. Initially, we attempted to solve this problem by using Mean-normalized Absolute Error loss. This was outperformed by another attempt using Mean Absolute Percentage Error (MAPE):

$$\mathrm{MAPE} = \frac{1}{|\mathbf{V}|} \sum_{v \in \mathbf{V}} \left| \frac{\mathbf{y}_v - \hat{\mathbf{y}}_v}{\mathbf{y}_v} \right| \tag{1}$$

which El Sayed et al note as a useful regression loss function for scale-invariance [4]. The MAPE Loss divides out the label from the absolute error in order to give a relative scale for the loss, which makes it an ideal choice for a simplistic loss function.

However, this approach was still insufficient. One reason for this could be that the smaller currents also have less variance than other currents by approximately 2 orders of magnitude. While this is considerably less than the 5 or more orders of magnitude in absolute value, this still means that it will contribute substantially less to the relative error loss function on average. In future work, we could attempt to solve this using a logarithmic scale for currents.

3.4. Batching

Because graphs may have arbitrary dimensions in their descriptive matrices, standard batching can be difficult. To fix this problem, a batch is instead merged into one effective graph, with the feature matrices concatenated vertically and the adjacency matrices diagonally within a larger square zero matrix. [1] At low batch sizes we observed typical problems with stability, but batching more than 32 graphs tends to cause a notable increase in measured error.

3.5. Feature Engineering

To manage the features, we used two main strategies: 1) Convert descriptive parameters to numerical encodings, using their implied ordinality: Skew was converted to a two-dimensional parameter describing the corner placement, and option was converted to a one-dimensional parameter encoding low, medium, or high power. The PVT parameters were dropped due to redundancy with other numerical parameters. 2) For imbalance in feature scale, we standardized features across the entire dataset to a mean of zero and variance of one. This helped to prevent cases of exploding gradient during training.

3.6. Implementation

Our training algorithm was implemented in Python, using the PyG library implementation for our GCN convolution layers [1]. The update rule for a feature vector $\mathbf{X}$ in this implementation is given by:

$$\mathbf{X}' = \hat{\mathbf{D}}^{-1/2} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-1/2} \mathbf{X} \mathbf{\Theta} \tag{2}$$

Where $\hat{\mathbf{A}}$ denotes the adjacency matrix with additional self-loops, $\hat{\mathbf{D}}$ denotes the degree matrix of $\hat{\mathbf{A}}$, and $\mathbf{\Theta}$ parameterizes the convolution filters. [8]

Our code repository can be found here: `https://github.com/YosemiteSamurai/CurrentPrediction`

4. Experiment

The goal of our experiment is primarily to verify the premise that a GCN can be trained for current prediction. We first train the network on a split of Dataset 2, then evaluate for relative error performance on the remaining split. Additionally, we compare the effectiveness of network depth, batch size, and graph models in overall network performance.

4.1. Design

Our experiment is designed as a simple grid-search performance comparison between several different versions of our GCN. To simplify comparison, layer width and learning rate are frozen at their respective best-performing levels for a configuration.

This will first aim to answer the preliminary question of whether any configuration can accurately predict currents in our test circuit. The comparison between runs can then be used to identify the best-performing combination of architecture and graph model.

4.2. Evaluation

Because we will want to know whether the target current is under a certain threshold, we will evaluate our results with relative error in order to know the confidence in meeting that threshold. In this case, the mean relative error differs slightly from the MAPE loss in that it is evaluated across the entire input set, rather than an average of an average batch-loss. We also track the maximum relative error to determine whether there are major outliers in the mean calculation. These metrics are somewhat approximative, and could potentially be replaced by a better fit evaluation metric such as the coefficient of determination [4] in future work.

4.3. Results

4.3.1 Depth Comparison

Depth	Loss	Mean RE	Max RE
2	1.03	1.39	26.3
3	.988	1.00	3.43
4	.976	.954	3.77

The fourth convolution layer must be stabilized with a much lower learning rate, but performance was still approximately even with the 3-layer network. With no reason in particular to use more layers, we assume the 3-layer network as our baseline for other tests, which has a noticeable improvement in both error metrics over 2 layers.

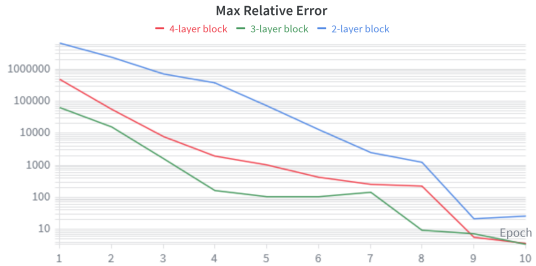

Figure 6: Depth Performance (log-scaled)

4.3.2 Batch Size Comparison

Batch Size	Loss	Mean RE	Max RE
8	1.29	2.18	34.3
16	.983	2.37	39.2
32	.988	1.00	3.43
64	.966	1.03	28.7
128	2.30	2.15	3120

A batch size of 32 was required for stable training. However, increasing beyond this point negatively impacts performance. The effect on the maximum outlier appears to be particularly drastic.

Figure 7: Batch Training Convergence

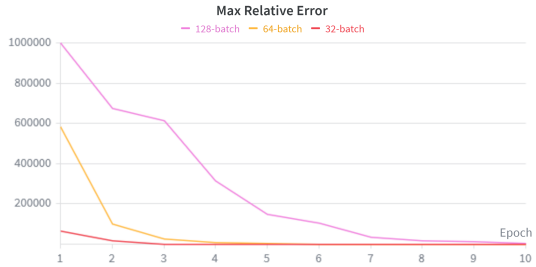

Figure 8: Batch Training Performance

4.3.3 Graph Comparison

Graph	Loss	Mean RE	Max RE
block	.988	1.00	3.43
split	2.52	2.21	17.5

The inverter-block graph consistently outperformed the split-transistor graph, which is surprising given the relatively minor difference in structure. This could indicate that more graphs should be added to the experiment.

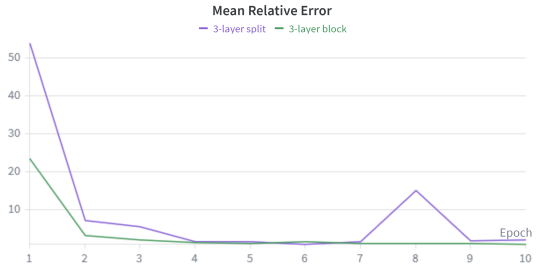

Figure 9: Graphical Model Performance

5. Conclusion

Overall, while our experiment results do not show that our network is sufficiently accurate, the convergence of our training algorithm suggests that, predicting current from a netlist and graph structure could still be possible with further performance improvements.

Our results also indicate that even in a controlled environment, graph structure can impact training performance. This could be due to a difference in current-scale distribution between the models we used, but it potentially indicates that it is worth investigating.

5.1. Future work

A key aspect that our experiment is missing is proof of our network's ability to generalize to different circuit designs. The introduction of arbitrarily long inverter chains to the training data would be a good next step. This would be a controlled environment with a known repeated component pattern, which could be conducive to convolution as well as potentially easier to automate simulation and model creation, which were limiting factors in the scope of our experiment.

Once it is known that the process works for more circuit designs, it may be useful to redo the experiment using a more advanced form of GNN, such as a Graph Attention Network, or look to other literature for pooling layer strategies.

There are also several other circuit-to-graph modeling strategies that could be tried. Physically accurate graphs could be constructed via the use of semi-supervised learning with unlabeled edges, or by using simple circuit formulae to approximate the remaining edges in some cases. Hypergraphs have also been researched for GNNs [15], which could solve the issue of translating junction wires to graph edges.

References

[1] Pyg documentation, 2026. [Online; accessed March 18, 2026]. URL: https://pytorch-geometric.readthedocs.io/en/latest/index.html.

[2] Donghyeok Cho, Taekwang Jang, and SeongHwan Cho. A fully integrated 4:1 dc–dc converter with series transmission and parallel reception through electromagnetically coupled class-d lc oscillators. *IEEE Journal of Solid-State Circuits*, 60(10):3464–3473, Oct 2025. doi:10.1109/JSSC.2025.3566479.

[3] Qing Dong, Mahmut E. Sinangil, Burak Erbagci, Dar Sun, Win-San Khwa, Hung-Jen Liao, Yih Wang, and Jonathan Chang. 15.3 a 351tops/w and 372.4gops compute-in-memory sram macro in 7nm finfet cmos for machine-learning applications. In *2020 IEEE International Solid-State Circuits Conference - (ISSCC)*, pages 242–244, Feb 2020. doi:10.1109/ISSCC19947.2020.9062985.

[4] Ziad El Sayed, Zeng Wang, Hana Selmani, Johann Knechtel, Ozgur Sinanoglu, and Lilas Alrahis. Graph neural networks for integrated circuit design, reliability, and security: Survey and tool. *ACM Comput. Surv.*, 58(4), October 2025. doi:10.1145/3769081.

[5] Luke R. Everson, Muqing Liu, Nakul Pande, and Chris H. Kim. An energy-efficient one-shot time-based neural network accelerator employing dynamic threshold error correction in 65 nm. *IEEE Journal of Solid-State Circuits*, 54(10):2777–2785, Oct 2019. doi:10.1109/JSSC.2019.2914361.

[6] Kathleen et al Feng. Amber: A 16-nm system-on-chip with a coarse- grained reconfigurable array for flexible acceleration of dense linear algebra. *IEEE Journal of Solid-State Circuits*, 59(3):947–959, March 2024. doi:10.1109/JSSC.2023.3313116.

[7] Himanshu Kaul, Mark A. Anders, Sanu K. Mathew, Gregory Chen, Sudhir K. Satpathy, Steven K. Hsu, Amit Agarwal, and Ram K. Krishnamurthy. 14.4 a 21.5m-query-vectors/s 3.37nj/vector reconfigurable k-nearest-neighbor accelerator with adaptive precision in 14nm tri-gate cmos. In *2016 IEEE International Solid-State Circuits Conference*

(ISSCC), pages 260–261, Jan 2016. doi:10.1109/ISSCC.2016.7418006.

[8] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *CoRR*, abs/1609.02907, 2016. URL: http://arxiv.org/abs/1609.02907, arXiv:1609.02907.

[9] Microelectronics Research Center, University of Minnesota. Predictive technology model (ptm). https://mec.umn.edu/ptm/#h.v5g6eyuivo9x, n.d. Accessed: 2026-03-16.

[10] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nova, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. A graph placement methodology for fast chipdesign. *Nature*, 594(7862):207–212, Jun 2021. doi:10.1038/s41586-021-03544-w.

[11] Myeong-Jae et al Park. A 192-gb 12-high 896-gb/s hbm3 dram with a tsv auto-calibration scheme and machine-learning-based layout optimization. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 65, pages 444–446, Feb 2022. doi:10.1109/ISSCC42614.2022.9731562.

[12] Myeong-Jae et al Park. A 192-gb 12-high 896-gb/s hbm3 dram with a tsv auto-calibration scheme and machine-learning-based layout optimization. *IEEE Journal of Solid-State Circuits*, 58(1):256–269, Jan 2023. doi:10.1109/JSSC.2022.3193354.

[13] Mahmut E. Sinangil, Yen-Ting Lin, Hung-Jen Liao, and Jonathan Chang. A 290-mv, 7-nm ultra-low-voltage one-port sram compiler design using a 12t write contention and read upset free bit-cell. *IEEE Journal of Solid-State Circuits*, 54(4):1152–1160, April 2019. doi:10.1109/JSSC.2019.2895236.

[14] Chenyang Zhao, Jinbei Fang, Xiaoli Huang, Deyang Chen, Zhiwang Guo, Jingwen Jiang, Jiawei Wang, Jianguo Yang, Jun Han, Peng Zhou, Xiaoyong Xue, and Xiaoyang Zeng. A 28-nm 36 kb sram cim engine with 0.173 m2 4t1t cell and self-load-0 weight update for ai inference and training applications. *IEEE Journal of Solid-State Circuits*, 59(10):3277–3289, Oct 2024. doi:10.1109/JSSC.2024.3399615.

[15] Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Graph neural networks: A review of methods and applications. *CoRR*, abs/1812.08434, 2018. URL: http://arxiv.org/abs/1812.08434, arXiv:1812.08434.

[16] Jonathan Zhou, Emir Ali Karahan, Sherif Ghozzy, Zheng Liu, Hossein Jalili, and Kaushik Sengupta. 25.3 ai-enabled design space discovery and end-to-end synthesis for rfics with reinforcement learning and inverse methods demonstrating mm-wave/sub-thz pas between 30 and 120ghz. In *2025 IEEE International Solid-State Circuits Conference (ISSCC)*, volume 68, pages 1–3, Feb 2025. doi:10.1109/ISSCC49661.2025.10904600.